Développement d'Applications Réparties

Chapitre 1 : Les sockets (3)

Séance 4 : 16/10/2026

Serveur multithreads - sockets TCP- Numéro de client

Création d'un Serveur

```
public class Server extends Thread{
public void run() {

/*La première étape : création de
l'objet ServerSocket*/

ServerSocket ss = new ServerSocket(2026);

 /* La deuxième étape : atttente de
 connexion de plusieurs clients
 Et création de l'objet Socket côté
 serveur suite à la connexion de chaque
 client*/
 int numCli=0
while (true) {
Socket s = ss.accept();
++numCli;
new Conversation(s,numCli).start();

// La dernière étape : Fermer socket
s.close();
}
```

```
public class Conversation extends Thread{
Socket s;
int numCli;
public Conversation(Socket socket,int n)
{
     super();
     this.s=socket;
     this.numCli=n;}
     // Traitement
     @Override
     public void run() {
          S.o.p(" ceci est un client numéro
          "+numCli);
          // La dernière étape : Fermer
          socket
          try {
          s.close();
          } catch (IOException e) {
          // e.printStackTrace();}
     }

}
   public static void main(String[] args){
   new Server().start();
   }
   }// Fin de la classe Server
```

Création d'un client

```
public static void main(String[] args){

/*La première étape : création de
 l'objet Socket côté client*/

Socket s = new Socket("localhost",2026);

S.o.p ("je suis un client connecté");

 // échange des données




 // La dernière étape : fermefla socket
    s.close();
```


© L. Bayoudhi DAR

Sockets en mode non connecté : principe

Mode datagramme

- Envois de paquets de données (datagrammes) indépendants sur le réseau.
- Pas de connexion entre parties client et serveur
- Pas de fiabilité ou de gestion de la communication
  - ✓ Un paquet peut ne pas arriver (perdu par le réseau)
  - ✓ Un paquet P2 envoyé après un paquet P1 peut arriver avant ce paquet P1 (selon la gestion des routes dans le réseau)
- La communication se fait souvent dans les 2 sens
  - ✓ Le serveur doit donc connaître la localisation du client
  - ✓ Elle est précisée dans le paquet qu'il reçoit du client (un paquet = Informations d'entête + données à transmettre)
  - ✓ le serveur peut répondre en envoyant des données à cette adresse, sans avoir besoin d'établir une connexion préalable.

Sockets en mode non connecté : principe

Client

1. La partie cliente crée une socket pour accéder à la couche UDP et la lie sur un port quelconque.
2. Le client envoie un paquet via sa socket en précisant l'adresse du destinataire

- Destinataire = partie serveur
  - **@IP de la machine** sur laquelle tourne la partie serveur et **numéro de port** sur lequel est liée la socket de la partie serveur.

Serveur

1. La partie serveur crée une socket et la lie à un port UDP particulier
2. Le serveur se met en attente de réception de paquet sur sa socket
3. Le paquet est reçu par le serveur (sauf pb réseau)
4. Lit et/ou écrit sur la nouvelle socket
5. Envoie de la réponse.

Les sockets UDP en Java

- **Classes du package java.net utilisées pour communication via UDP**
  - ►**InetAddress :** codage des adresses IP.
  - ►**InetSocketAddress :** codage du couple @IP/port.
  - ►**DatagramSocket** : socket mode non connecté (UDP)
  - ►**DatagramPacket** : paquet de données envoyé via une socket mode non connecté (UDP)

Les sockets UDP en Java

- **Classes du package java.net utilisées pour communication via UDP**

  ►**InetSocketAddress :** implémente une adresse IP de socket, c'est-à-dire, une adresse IP et un numéro de port ou nom d'hôte et numéro de port.

  ❖Principalement utilisée pour identifier le destinataire ou la source dans une connexion réseau.

  - **Constructeurs :**

    ❑ InetSocketAddress(InetAddress addr, int port) : crée une adresse socket à partir de l'adresse IP et d'un numéro de port.

    ❑ InetSocketAddress(String hostname, int port) : crée une adresse socket à partir d'un nom d'hôte et un numéro de port.

Les sockets UDP en Java

- **Classes du package java.net utilisées pour communication via UDP**

  ►**InetSocketAddress :** implémente une adresse IP de socket, c'est-à-dire, une adresse IP et un numéro de port ou nom d'hôte et numéro de port.

  - **Méthodes**

    - ❑ InetAddress getAddress(): obtient l'objet InetAddress.
    - ❑ String getHostName() : obtient le nom d'hôte.
    - ❑ int getPort() : obtient le numéro de port.

Les sockets UDP en Java

- **Classes du package java.net utilisées pour communication via UDP**
  - ►**DatagramSocket** : socket mode non connecté
  - **Constructeurs :**
    - ❑ public DatagramSocket() throws SocketException : crée une nouvelle socket en la liant à un port quelconque libre
      - ▬ Exception levée en cas de problème
    - ❑ public DatagramSocket(int port) throws SocketException : crée une nouvelle socket en la liant au port local précisé par le paramètre port.

Les sockets UDP en Java

- **Classes du package java.net utilisées pour communication via UDP**

  ► **DatagramSocket** : socket mode non connecté

  - **Méthodes**

    ✓ Emission/réception de données : on récupère les paquets sur la socket à travers :

    ❑ void send(DatagramPacket p) throws IOException : Envoie le paquet passé en paramètre. Le destinataire est identifié par le couple @IP/port précisé dans le paquet.

    — Exception levée en cas de problème d'entrée/sortie

    ❑ void receive(DatagramPacket p) throws IOException : Reçoit un paquet de données passé en paramètre.

    - Les données reçues sont copiées dans le tableau passé en paramètre lors de la création de ***p***.
    - Les attributs d'@IP et de port de ***p*** contiennent l'@IP et le port de la socket distante qui a émis le paquet.

Les sockets UDP en Java

- **Classes du package java.net utilisées pour communication via UDP**
  - ►**DatagramPacket** : Structure des données en mode datagramme
  - **Constructeurs :**
    - ❑ public DatagramPacket(byte[] buf, int length) : Création d'un paquet pour **recevoir** des données (sous forme d'un tableau d'octets).
      - − Les données reçues seront placées dans ***buf*** qui doit être assez grand pour recevoir ce paquet.
      - − ***length*** précise la taille max de données à lire
        - − Ne pas préciser une taille plus grande que celle du tableau.
        - − En général, ***length*** = taille de ***buf***

Les sockets UDP en Java

- **Classes du package java.net utilisées pour communication via UDP**
  - ►**DatagramPacket** : Structure des données en mode datagramme
  - **Constructeurs :**
    - ❑ public DatagramPacket(byte[] buf, int length, InetAddress address, int port)
    - ❑ Création d'un paquet pour **envoyer** des données (sous forme d'un tableau d'octets)
      - ***buf*** : contient les données à envoyer
      - ***length*** : longueur des données à envoyer
      - Ne pas préciser une taille supérieure à celle de ***buf***
      - ***address***: adresse IP de la machine destinataire des données
      - ***port*** : numéro de port distant (sur la machine destinataire) où envoyer les données

Les sockets UDP en Java

- **Classes du package java.net utilisées pour communication via UDP**

  ► **DatagramPacket** :

  - **Méthodes**

    ❑ InetAddress getAddress() :

    - si paquet à envoyer : adresse de la machine destinataire
    - si le paquet reçu : adresse de la machine qui a envoyé le paquet (expéditeur).

    ❑ int getPort() :

    - si le paquet à envoyer : port destinataire sur la machine distante
    - et si le paquet reçu : port utilisé par le programme distant pour envoyer le paquet.

    ❑ byte[] getData() : données contenues dans le paquet.

    ❑ int getLength() :

    - si le paquet à envoyer : longueur des données à envoyer
    - et si le paquet reçu : longueur des données reçues.

Principe d'une application C/S avec les sockets en mode non connecté

Création d'un Serveur

public static void main(String[] args){

```
//création d'une socket liée au port 2026
DatagramSocket socket = new DatagramSocket(2026);
//tableau de 1024 octets qui contiendra les données
reçues
byte[] buffer=new byte[1024];
String donnees = "";
String message = "";
int taille = 0;
System.out.println("Lancement du serveur");
 while (true) {
//création d'un paquet en utilisant le tableau d'octets
DatagramPacket paquet = new DatagramPacket(buffer, buffer.length);
DatagramPacket envoi = null;
//attente de la réception d'un paquet.Le paquet reçu est
 //placé dans packet et ses données dans donnees.
socket.receive(paquet);
System.out.println("\n"+paquet.getAddress());
taille = paquet.getLength();
//récupération des données (une chaîne de caractères)
donnees = new String(paquet.getData(),0, taille);
System.out.println("Donnees reçues = "+donnees);
message = "Bonjour "+donnees+ " !";
System.out.println("Donnees envoyees = "+message);
envoi = new DatagramPacket(message.getBytes(),message.length(),
paquet.getAddress(),paquet.getPort());
socket.send(envoi);}}
```

Création d'un client

public static void main(String[] args){

```
String ch= "LCS3-GLSI";
int length = ch.length();
//données à envoyer
byte[] buffer = ch.getBytes();
//création d'une socket, sans la lier à un port particulier
DatagramSocket socket = new DatagramSocket();
//création du paquet l'adresse du serveur
InetSocketAddress ia=new InetSocketAddress("localhost",2026);

        DatagramPacket donneesEmises = new
        DatagramPacket(buffer,length, ia);
        //envoi du paquet via la socket
        socket.send(donneesEmises);

DatagramPacket donneesRecues = new DatagramPacket(new byte[1024],
1024);
socket.receive(donneesRecues);
System.out.println("Message : " + new String(donneesRecues.getData(), 0,
donneesRecues.getLength()));
System.out.println("de : " + donneesRecues.getAddress() + ":" +
donneesRecues.getPort());
}
```

Choix entre UDP et TCP

✓ A priori simple

- TCP est fiable

✓ UDP :

- Non fiable
- Attention à la taille réservée : si le récepteur réserve un tableau trop petit par rapport à celui envoyé, une partie des données est perdue

✓ Mais intérêt tout de même pour UDP dans certains cas :

- Si la fiabilité n'est pas essentielle
- Si la connexion entre les 2 applications n'est pas utile

❑ Exemple :

- Un thermomètre envoie toutes les 5 secondes la température de l'air ambiant à un afficheur distant
- Pas grave de perdre une mesure de temps en temps
- Pas grave d'envoyer les mesures même si l'afficheur est absent